Optiver 项目文档

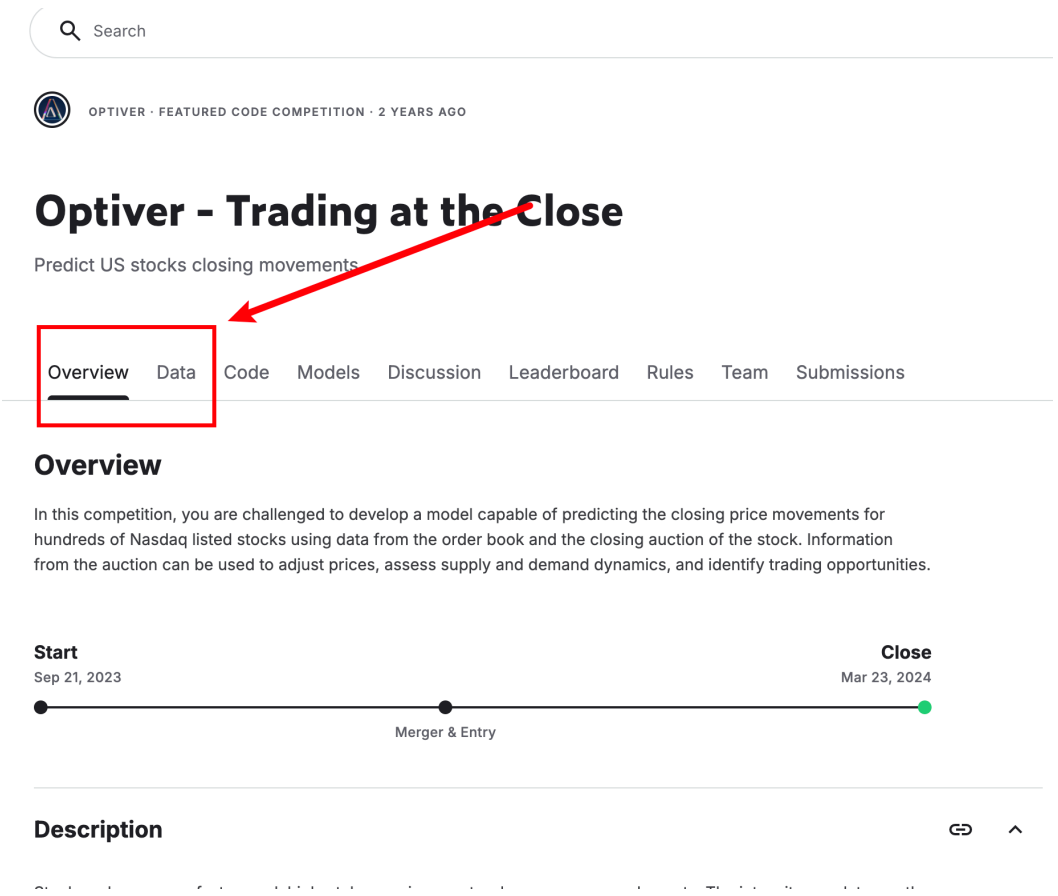
1 比赛任务

在这个比赛中，我们的任务是开发一个模型，能够使用来自股票订单簿和收盘拍卖的数据来预测数百个纳斯达克上市股票的收盘价变动。拍卖中的信息可用于调整价格、评估供需动态，并识别交易机会。开放的模型可以帮助整合来自拍卖和订单簿的信号，从而提高市场效率，特别是在交易的最后十分钟的激烈时刻。这些问题类似于 Optiver 的交易员、定量研究员和工程师面临的问题。

所以这是一个回归问题 (regression)，比赛评判指标用的是 Mean Absolute Error (MAE)：

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - x_i|$$

要看简介和数据



2 背景知识

纳斯达克股票交易所的每个交易日都以**纳斯达克收盘拍卖**结束。这个过程确定了在交易所上市证券的官方收盘价格。这些收盘价格作为投资者、分析师和其他市场参与者评估个别证券和整个市场表现的关键指标。纳斯达克平均每日交易量的**近 10% 发生在收盘拍卖中**。提供真实的价格和规模发现，收盘拍卖为指数基金和其他投资策略确定基准价格。

2.1 收盘拍卖

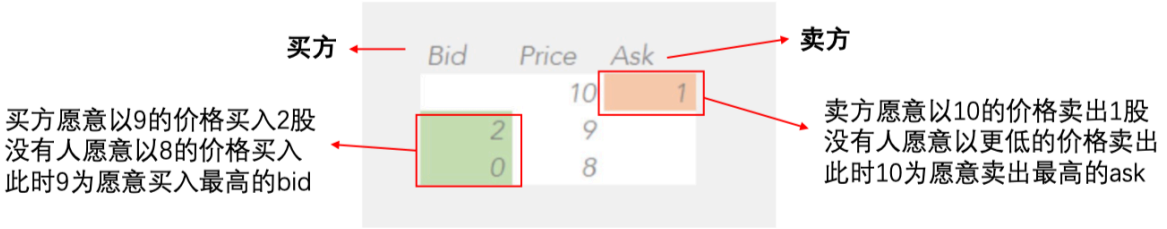
在金融市场中，拍卖是一种确定特定资产价格的机制，它允许多个买家和卖家在一个受控制、规范的环境中直接互动。对于这次比赛，我们的 focus 是收盘拍卖。在收盘拍卖中，订单在预定的时间内被收集，然后以拍卖参与者表达的买卖需求确定的单一价格进行匹配。对于纳斯达克收盘拍卖，交易所从交易日开始就接受订单，并从东部时间下午 3:50 开始，为期 **10 分钟**公布拍卖簿的状态，直到下午 4 点市场关闭时刻，此时订单会立即以单一价格匹配。

收盘价的确定方式如下： 能够**匹配最多数量股份的价格**。如果拍卖在不同价格层次上有相等数量的匹配批次，纳斯达克会使用一种专有算法来决定价格，这种算法考虑了最后交易价格、订单的价格-时间优先以及不同价格层次上可用的流动性。能够匹配最大数量股份的价格不是唯一的情况相当罕见。

2.2 order book（订单薄）

订单簿”一词指的是按价格层次组织的特定证券或金融工具的买入（也称为买价）和卖出（也称为卖价）订单的电子列表。

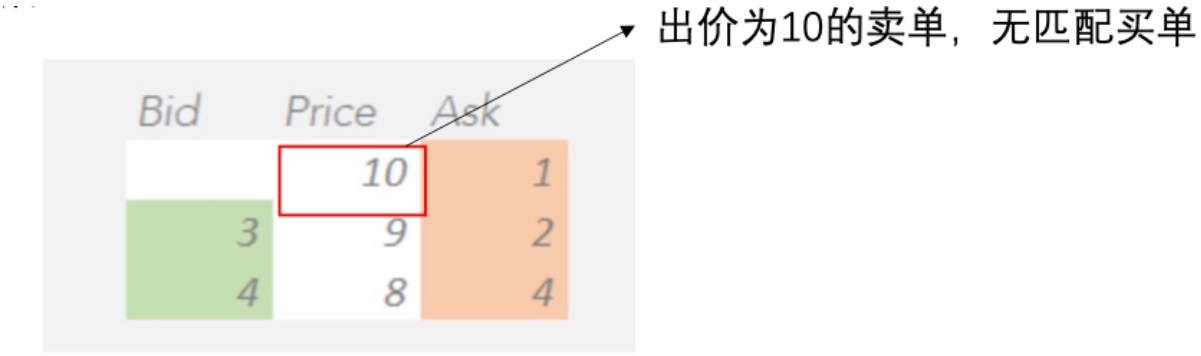
以下是一个在连续（非拍卖）交易中的订单簿的简单示例。我们可以看到，在9的价格层次上，买价方有2股，意味着市场参与者愿意以9的价格购买2股。同样，我们看到在10的价格层次上，卖价方有1股，意味着市场参与者愿意以10的价格出售1股。



在连续交易的订单簿中，最高的买价总是低于最低的卖价，也叫做bid-ask spread。例如，如果有人提出以9元的价格卖出10股的挂单，那么会有2股被匹配，因有人愿意以每股9元的价格买入2股。新的最佳卖价会变成以9元价格的8股，订单簿就会更新

2.3 auction order book（拍卖订单薄）

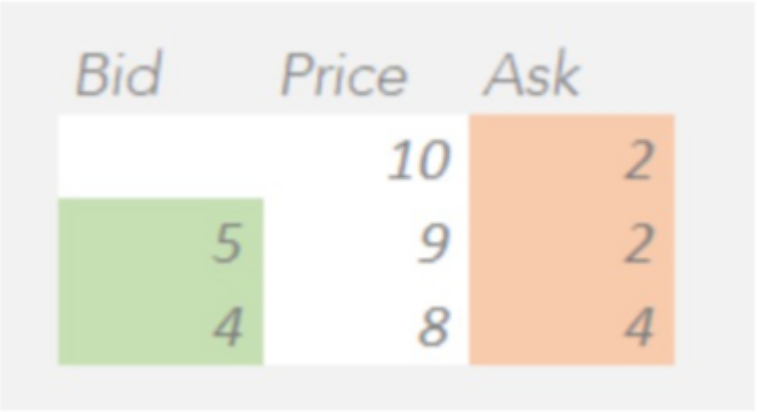
拍卖订单簿的行为方式与持续交易的订单簿略有不同。在这个簿子中， 订单不会立即匹配，而是收集起来，直到拍卖结束的那一刻。在下面的例子中可以注意到，与持续交易的订单簿不同，最高出价并不高于最低要价。由于最佳买入和卖出价重叠，所以下面例子中的簿子被称为交叉。因此，收盘拍卖价格被称为解交叉价格，即能够匹配那些处于交叉状态的股票的价格。



当价格为10时，将不会有任何批次匹配，因为没有出价>=10的出价。当价格为9时，将匹配3个批次，因为有3个出价>=9，且有6个要价<=9。当价格为8时，将匹配4个批次，因为有7个出价>=8，且有4个要价<=8。因此，能够最大化匹配批次数量的价格是8。我们会以以下方式描述拍卖订单簿：交叉价格为8；匹配的大小为4个批次。买方方向的不平衡为3个批次。术语“不平衡"指的是未匹配的股份数量。在解交叉价格8的情况下，有7个出价和4个要价可以匹配，因此我们剩下3个未匹配的出价。由于出价是买单，所以买方方向上有3个批次的不平衡。“远端价格”这一术语指的是拍卖订单簿的假设性解交叉价格，如果它在报告时间解交叉的话。纳斯达克在收盘交叉前5分钟提供远端价格信息。

2.4 将两个订单簿结合起来

可以更准确地反映市场在不同价格水平上的买卖兴趣。这有助于更好的价格发现，允许市场在拍卖解交叉时达到更准确和公平的均衡价格。在这里，我们通过聚合所有价格水平上的买卖兴趣，结合了我们最初的订单簿示例和我们的拍卖簿示例。



Bid	Price	Ask
	10	2
5	9	2
4	8	4

对于这个组合的订单簿：

当价格为10时，将不会有任何批次匹配，因为没有出价>=10的出价。当价格为9时，将匹配5个批次，因为有5个出价 >=9，且有6个要价<=9。当价格为8时，将匹配4个批次，因为有9个出价>=8，且有4个要价<=8。因此，能够最大化匹配批次数量的价格是9。我们会以以下方式描述组合订单簿：交叉价格为9；匹配的大小为5个批次；不平衡为1个批次，方向为卖方。

2.5 其他术语定义

Far Price（远端价格）是在拍卖结束前，交易所提供的一个价格信号，指的是如果拍卖此刻结束，预计将成交的价格。这个价格反映了当前的买卖订单状况，但由于实际拍卖还未结束，因此被称为"远端"。纳斯达克等交易所会在拍卖结束前的几分钟提供这个价格，帮助市场参与者做出更好的交易决策。

Near Price（近端价格）与远端价格类似，但更接近实际拍卖结束时的成交价格。这个价格是在拍卖结束前不久由交易所提供的，基于此时的订单簿来预估的可能成交价格。近端价格比远端价格提供了一个更新、更接近最终成交价格的估计。

纳斯达克还提供一个公平价格的指示，称参考价格。参考价格的计算方法如下：

- 如果近端价格位于最佳买入价和卖出价之间，则参考价格等于近端价格
- 如果近端价格>最佳卖出价，则参考价格=最佳卖出价
- 如果近端价格＜最佳买入价，则参考价格=最佳买入价因此，参考价格是近端价格，限定在最佳买入价和卖出价之间。

2.6 更详细的背景知识

股票交易

1. https://www.investopedia.com/terms/b/bid-askspread.asp
2. https://en.wikipedia.org/wiki/Bid%E2%80%93ask_spread
3. https://www.cmcmarkets.com/en/trading-guides/trading-strategies
4. https://medium.com/the-investors-handbook/17-basic-stock-market-concepts-you-need-to-know-8af22ae3c016
5. https://www.nseindia.com/learn/online-recorded-courses-trading-strategies
6. https://www.investopedia.com/terms/o/ohlcchart.asp
7. https://www.sciencedirect.com/topics/economics-econometrics-and-finance/market-microstructure
8. https://en.wikipedia.org/wiki/Market_microstructure
9. https://quantitativebrokers.com/blog/what-is-market-microstructurenbsp

做市

1. https://blog.quantinsti.com/market-making/
2. https://www.investopedia.com/terms/m/marketmaker.asp
3. https://www.investopedia.com/ask/answers/06/brokerandmarketmaker.asp

订单类型和订单簿

1. https://www.investopedia.com/terms/o/order-book.asp
2. https://www.investopedia.com/investing/basics-trading-stock-know-your-orders/
3. https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders
4. https://www.schwab.com/learn/story/3-order-types-market-limit-and-stop-orders
5. https://www.simtrade.fr/blog_simtrade/understanding-the-order-book-how-it-impacts-trading/

拍卖价格决策框架

1. https://www.investopedia.com/terms/a/auctionmarket.asp
2. https://www.investopedia.com/articles/investing/091113/auction-method-how-nyse-stock-prices-are-set.asp

3. https://www.motilaloswal.com/blog-details/what-is-auction-and-what-is-the-process-of-that/21292
4. https://www.nyse.com/article/parity-priority-explainer
5. https://www.investopedia.com/terms/m/matchingorders.asp

3 数据概览

在这场比赛中，我们的任务是预测10分钟拍卖期间的短期价格变动。原始数据包括特征列如下：

stock_id：股票的唯一标识符。date_id- 日期的唯一标识符

imbalance_size：当前参考价格下未匹配的金额（以美元计）
imbalance_buy_sell_flag - 反映拍卖不平衡方向的指标。

reference_price, matched_size, far_price, near_price

[bid/ask]_price：非拍卖簿中最具竞争力的买入/卖出价格。

[bid/ask]_size：非拍卖簿中最具竞争力的买入/卖出水平的美元名义金额。

wap：非拍卖簿中的加权平均价格。

seconds_in_bucket：自当日收盘拍卖开始以来经过的秒数，始终从0开始。

target：未来60秒内股票的wap变化，减去合成指数未来60秒的wap变化

	stock_id	date_id	seconds_in_bucket	imbalance_size	imbalance_buy_sell_flag	reference_price
0	0	0	0	3180602.69	1	0.999812
1	1	0	0	166603.91	-1	0.999896
2	2	0	0	302879.87	-1	0.999561
3	3	0	0	11917682.27	-1	1.000171
4	4	0	0	447549.96	-1	0.999532

matched_size	far_price	near_price	bid_price	bid_size	ask_price	ask_size	wap	target
13380276.64	NaN	NaN	0.999812	60651.50	1.000026	8493.03	1.0	-3.029704
1642214.25	NaN	NaN	0.999896	3233.04	1.000660	20605.09	1.0	-5.519986
1819368.03	NaN	NaN	0.999403	37956.00	1.000298	18995.00	1.0	-8.389950
18389745.62	NaN	NaN	0.999999	2324.90	1.000214	479032.40	1.0	-4.010200
17860614.95	NaN	NaN	0.999394	16485.54	1.000016	434.10	1.0	-7.349849

预测值

4 方案详细介绍

4.1 交叉验证方式

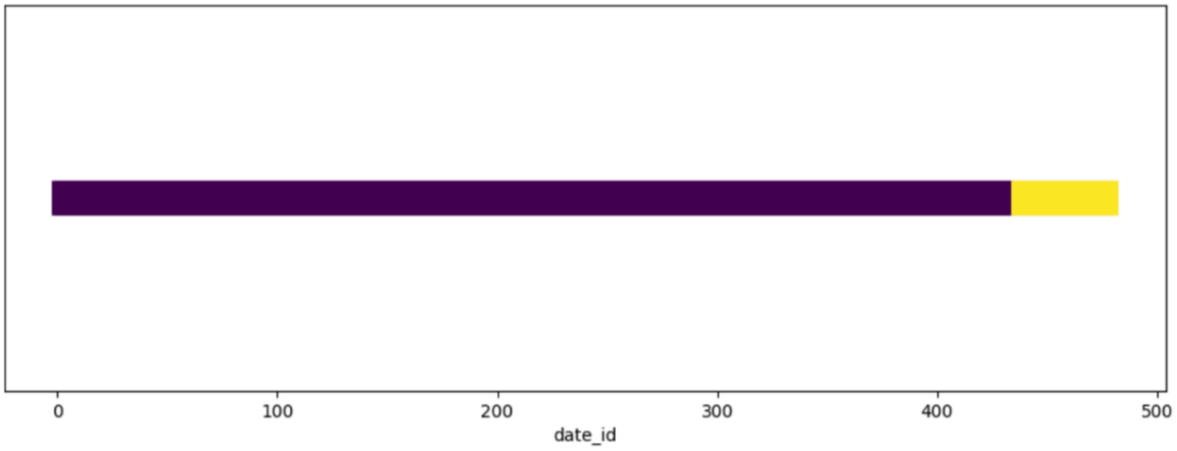
比赛给定的原始数据中仅包含了price，size，imbalance等基础信息，所以我们的第一步是先对这些基础特征进行特征工程来得到更丰富的特征。

但是并不是什么特征都可以提高模型表现，我们想要的是对模型表现有贡献的特征，这也是kaggle比赛的一根难点，就是如何找到"有用”的特征。一个简单的思路就是每加入一些特征之后就提交模型到leaderboard看 public leaderboard的得分变化，但这存在过拟合的风险，因为我们最后是用另外的数据集进行测试。一个更有效的办法是选择一种有效的交叉验证（cross validation）的策略。

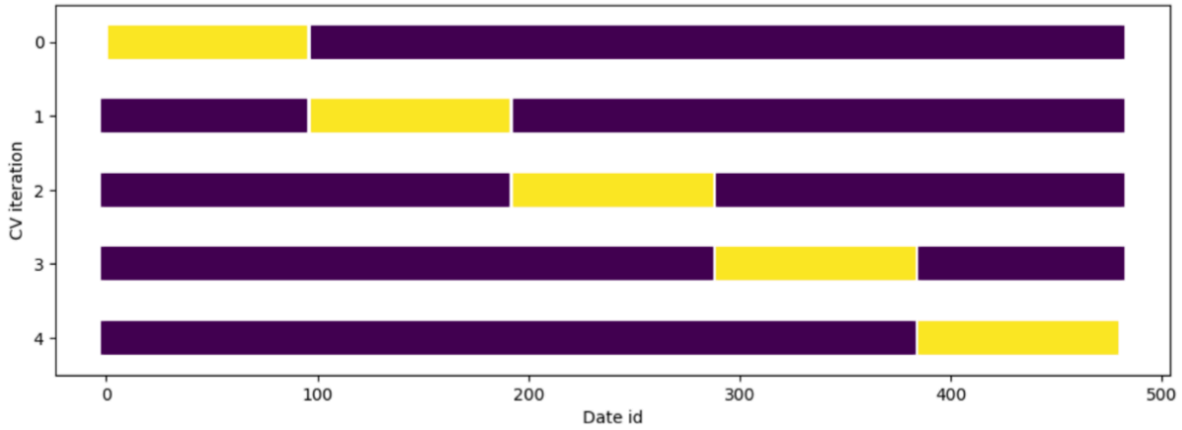
在开发出一个有效的交叉验证的策略之后，我们更关注的是交叉验证的分数，而不是一味的过拟合 leaderboard。除此之外，在本地进行交叉验证也可以有效的节省提交任务的时间。

因此，我们先介绍本次比赛方案选择的交叉验证方式：

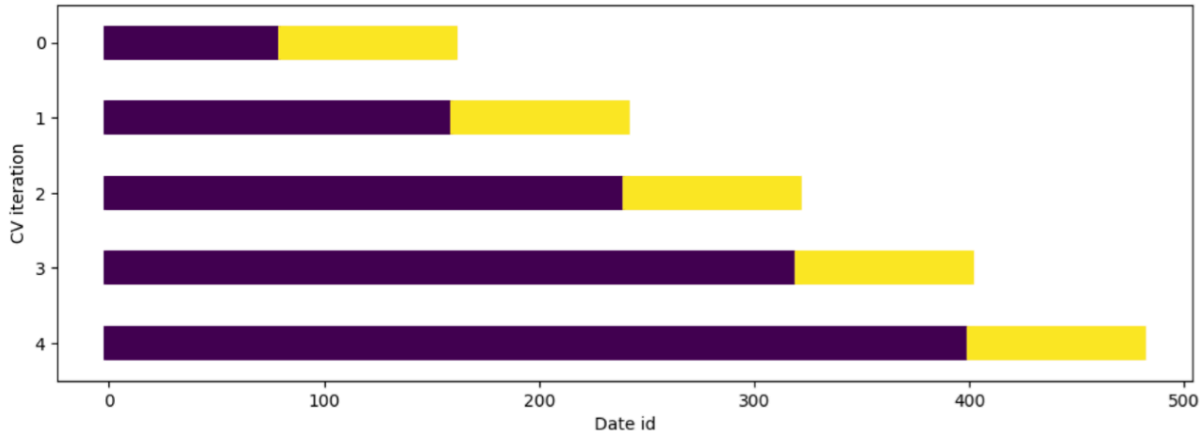
Hold-one-out CV：按照date_id进行训练集和验证集分割，留出一部分训练，另一部分作为验证。高效，方便，没有破坏数据的时间序 列特性，因此是一种合理的CV方式。



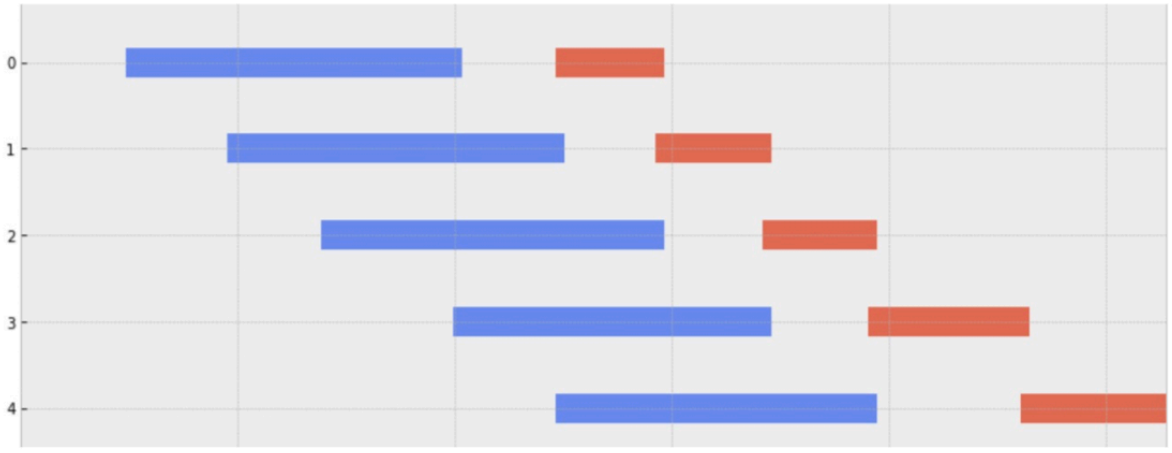
K-Fold CV：传统的CV方式，但是不适合本场比赛，因为破坏了时间序列特征（用未来的数据训练，在过去的数据上验证， 存在数据泄露的问题），因此不选择本种交叉验证策略。



时间序列CV：传统时间序列的交叉验证策略。适用于本场比赛。缺点是前几折只能用到一小部分的数据训练。



Purged Time Series CV：这种CV的好处是进一步减少了数据泄露的可能性，通过在训练集与验证集之间设置了一个gap



综上所述，本方案采取hold-one-out CV进行本地的特征与模型测试，因为快，方便模型提交采取purged time series CV因为最大程度上避免了数据泄露。

4.2 特征工程

4.2.1 领域特征

需要一定领域知识，通过组合变换基础的特征，比如bid price, ask price, imbalance size等，来获取更有意义的特征表示。

```
df["volume"] = df.eval("ask_size + bid_size")
df["mid_price"] = df.eval("(ask_price + bid_price) / 2")
df["liquidity_imbalance"] = df.eval("(bid_size-ask_size)/(bid_size+ask_size)")
df["matched_imbalance"] = df.eval("(imbalance_size-matched_size)/(matched_size+imbalance_size)")
df["size_imbalance"] = df.eval("bid_size / ask_size")
```

```
df["stock_weights"] = df["stock_id"].map(weights)
df["weighted_wap"] = df["stock_weights"] * df["wap"]
df['wap_momentum'] = df.groupby('stock_id')['weighted_wap'].pct_change(periods=6)
df["imbalance_momentum"] = df.groupby(['stock_id'])['imbalance_size'].diff(periods=1) / df['matched_size']
```

4.2.2 交互特征

在原始特征集上进行交互特征的生成，通过测试可以有效提高模型的学习能力。我们先是计算prices相关的特征两两之间的交互，具体定义如下：

```
for c in combinations(prices, 2):
    df[f"{c[0]}_{c[1]}_imb"] = df.eval(f"({c[0]} - {c[1]})/({c[0]} + {c[1]})")
```

除此之外，为了捕捉三个指标之间的相对关系和动态，我们也计算了部分price特征和sizes相关特征的三重交互特征，通过考虑组合的最大值（max_val）、最小值（min_val）和中间值（mid_val）之间的相对差异来实现

```
for c in [['ask_price', 'bid_price', 'wap', 'reference_price'], sizes]:
    triplet_feature = calculate_triplet_imbalance_numba(c, df)
    df[triplet_feature.columns] = triplet_feature.values
```

这里使用了Numba库，它是一个高性能的Python编译器，能够将Python函数编译成机器代码。这在处理大规模的数据和进行重复计算时，能够显著提高效率。具体操作见源代码。

4.2.3 排序特征

这些特征是通过在特定的时间点对某些关键指标进行排序来生成的。这些指标包括参考价格、交易量、中间价格和匹配的不平衡。每个特征都是在每个日期（date_id）和每个时间桶 （seconds_in_bucket）内的排序值。

```
df['reference_price_rank'] = df.groupby(['date_id', 'seconds_in_bucket'])['reference_price'].rank(method='average')
df['volume_rank'] = df.groupby(['date_id', 'seconds_in_bucket'])['volume'].rank(method='average')
df['mid_price_rank'] = df.groupby(['date_id', 'seconds_in_bucket'])['mid_price'].rank(method='average')
df['imbalance_rank'] = df.groupby(['date_id', 'seconds_in_bucket'])['matched_imbalance'].rank(method='average')
```

这里的rank（method='average）表示如果有多个相同的值，它们的排名会被设置它们排名值的平均数。

通过对这些关键指标进行排名，可以帮助揭示市场中不同股票之间的相对差异。

4.2.4 市场动态统计特征

这段代码为价格和交易量大小相关的列计算了一系列统计特征，包括平均值（mean)、标准差（std)、偏度（skew）和峰度（kurt)。每个统计函数对应的特征都是在每一行（即对于每个时间点和每个股票）上对所有价格或大小的数值计算得出的。这些特征能够提供有关数据分布的有用信息，揭示市场情绪的微妙变化和价格行为的内在动态。比如，高偏度可能表示一侧（上或下）有更多极端变化，而高峰度可能表示出现了异常值或市场出现波动的频率增加。

```
for func in ["mean", "std", "skew", "kurt"]:
    df[f"all_prices_{func}"] = df[prices].agg(func, axis=1)
    df[f"all_sizes_{func}"] = df[sizes].agg(func, axis=1)
```

4.2.5 滚动特征（重要）

下面我们介绍本方案里最重要的一些特征，也是提分的关键，就是融入了滚动计算的特征。

```
for col in ['matched_size', 'imbalance_size', 'reference_price', 'imbalance_buy_sell_flag']:
    for window in [1, 2, 3, 5, 8,10]:
        df[f"{col}_shift_{window}"] = df.groupby(['stock_id','date_id'])[col].shift(window)
        df[f"{col}_ret_{window}"] = df.groupby(['stock_id','date_id'])[col].pct_change(window)
```

这段代码关注于matched_ _size、imbalance. _size、reference_price、imbalance_buy_sel_flag，通过对每个股票ID和日期组合进行分组，计算了过去特定窗口期（1, 2，3, 5， 8, 10天）的位移（shift）和百分比变化（percentage change)。

这些特征旨在揭示短期和中期内这些指标的变动趋势，以及它们是如何随时间变化的。

```
for col in ['ask_price', 'bid_price', 'ask_size', 'bid_size',
            'wap','size_imbalance','market_urgency','imbalance_momentum','price_spread']:
    for window in [1, 2, 3, 5, 8,10]:
        df[f"{col}_diff_{window}"] = df.groupby(['stock_id','date_id'])[col].diff(window)
```

对于价格和交易量相关的指标（如ask_price, bid_price等），计算了过去特定窗口期内的差分值（diff），这些特征旨在捕捉价格和交易量在短期内的绝对变动。

4.2.6 长短期差异特征

启发于macd，通过计算10天和1天差分值的差异（例如wap_10_1），以及3天和1天差分值的差异（例如wap_3_1），这类特征旨在捕捉长期变动与短期变动之间的关系。这些差异可能揭示出市场趋势的转变或是短期波动的加剧。

```
#Long term - short term diff
for col in ['wap','market_urgency', 'imbalance_momentum', 'size_imbalance']:
  df[f"{col}_10_1"] =  df[f"{col}_diff_10"] - df[f"{col}_diff_1"]
  df[f"{col}_3_1"] =  df[f"{col}_diff_3"] - df[f"{col}_diff_1"]
```

4.2.7 价格和大小变动差异特征

创建了基于买价和卖价、买单量和卖单量差分值之间差异的特征（例如price_change_diff_1和size_change_diff_1），旨在从买卖两侧的角度揭示市场力量的不对称性和可能的价格压力方向。

4.2.8 滚动标准差差分特征

对ask_price、bid_price、ask_size、bid_size、wap的差分值计算了过去3天、5天和10天内的滚动标准差。这些特征旨在量化短期内市场变动的波动性。滚动标准差提供了一个时间窗口内数据波动大小的度量，高值可能指示市场不稳定或波动加剧。

```python
pl_df = pl.from_pandas(df)
windows = [3, 5, 10]
columns = ['ask_price', 'bid_price', 'ask_size', 'bid_size', 'wap']
group = ['stock_id','date_id']
expressions = []
```

```python
for window in windows:
    for col in columns:

        rolling_std_expr = (
            pl.col(f"{col}_diff_{window}")
            .rolling_std(window)
            .over(group)
            .alias(f'rolling_std_diff_{col}_{window}')
        )

        expressions.append(rolling_std_expr)
```

4.2.9 滚动最大值和最小值差异

计算wap在3和10窗口内的滚动最大值和最小值之差（例如wap_ptp_3）。这些特征反映了特定时间段内价格的波动范围， 可以作为市场波动性的一个指标。

```python
windows = [3, 10]
columns = ['wap']
group = ['stock_id','date_id']
expressions = []

for window in windows:
    for col in columns:
        rolling_max_expr = (
            pl.col(col)
            .rolling_max(window)
            .over(group)
            .alias(f'rolling_max_{col}_{window}')
        )

        rolling_min_expr = (
            pl.col(col)
            .rolling_min(window)
            .over(group)
            .alias(f'rolling_min_{col}_{window}')
        )

        expressions.append(rolling_max_expr)
        expressions.append(rolling_min_expr)
```

同时，我们又衍生出了wap在这两个窗口最大值与最小值的差值，捕捉短期内wap的波动幅度。

```python
for window in [3,10]:
  df[f"wap_ptp_{window}"] = df[f'rolling_max_wap_{window}'] - df[f'rolling_min_wap_{window}']
```

4.2.10 价格和大小变化差异的滚动标准差

对 price_change_diff 和 size_change_diff 在不同窗口 （3、5、10）内的滚动标准差进行了计算。这些特征旨在捕捉买卖双方力量变化的波动性和不确定性。

```python
#Rolling diff-diff
windows = [3, 5, 10]
columns = ['price_change_diff','size_change_diff']
group = ['stock_id','date_id']
expressions = []
```

```python
for window in windows:
    for col in columns:

        rolling_std_expr = (
            pl.col(f"{col}_{window}")
            .rolling_std(window)
            .over(group)
            .alias(f'rolling_std_diff_{col}_{window}')
        )

        expressions.append(rolling_std_expr)
```

4.2.11 时间特征

我们还提取了多个时间特征，为了捕捉不同时间段内的交易行为的细微差异。

```python
df["dow"] = df["date_id"] % 5
df["seconds"] = df["seconds_in_bucket"] % 60
df["minute"] = df["seconds_in_bucket"] // 60
df['time_to_market_close'] = 540 - df['seconds_in_bucket']
df['auction_stage'] = np.where(df['seconds_in_bucket']>290,1,0)
```

4.2.12 全局特征

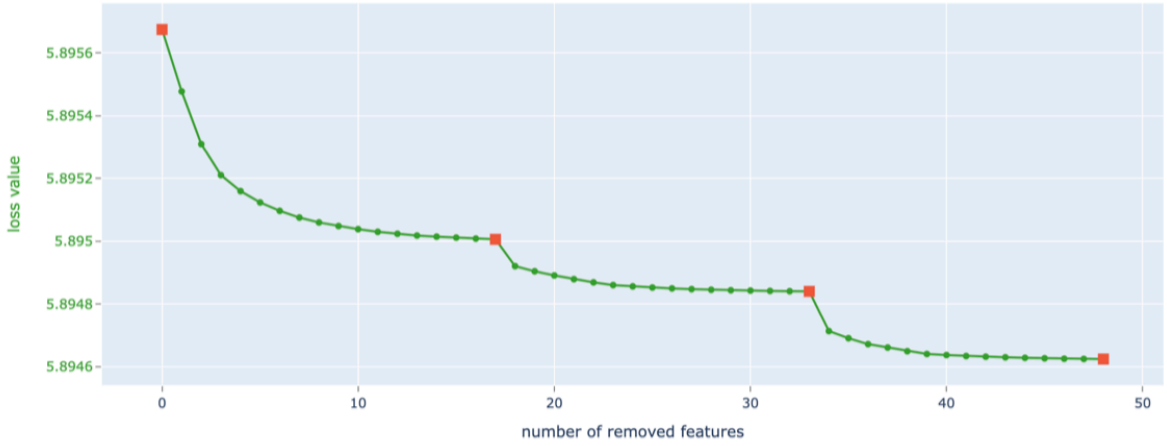
这些特征基于每只股票的交易量和价格，使用了中位数、标准差和极差（peak-to-peak）三种统计方法。这些特征反映了每只股票在整个训练数据集中的整体交易行为和价格波动性。但是注意这种特征是完全静态的，因此在使用的时候需要小心过拟合历史数据。

```python
global_stock_id_feats = {
    "median_size": df_train.groupby("stock_id")["bid_size"].median() + df_train.groupby("stock_id")["ask_size"].median(),
    "std_size": df_train.groupby("stock_id")["bid_size"].std() + df_train.groupby("stock_id")["ask_size"].std(),
    "ptp_size": df_train.groupby("stock_id")["bid_size"].max() - df_train.groupby("stock_id")["bid_size"].min(),

    "median_price": df_train.groupby("stock_id")["bid_price"].median() + df_train.groupby("stock_id")["ask_price"].median(),
    "std_price": df_train.groupby("stock_id")["bid_price"].std() + df_train.groupby("stock_id")["ask_price"].std(),
    "ptp_price": df_train.groupby("stock_id")["bid_price"].max() - df_train.groupby("stock_id")["ask_price"].min(),
```

4.2.13 最终的特征

随着特征的删除，loss下降，最终新特征及一共从原始特征集中保留了167个特征。



5 代码学习

5.1 方案

模型选择：使用了Xgboost，并且尝试了不同的随机种子和157个特征

特征工程：创建基于原始特征的“基础特征”，如加、减、乘、除。对交易量size特征进行中位数缩放处理。添加了不同时间窗口的差分、移动、滚动平均和标准差特征。利用历史WAP计算前6秒的目标，并进行滚动平均处理。创建了一些全局日期ID加秒数加权特征。从公开笔记本复制了市场紧急性和不平衡特征。应用MACD特征和针对stock_id加seconds_in_bucket的目标滚动平均。

特征选择：通过分组添加特征并检查本地交叉验证分数是否有所改进。在最终模型中保留了157 个特征。

后处理：从目标的定义出发，对所有股票的目标加权和进行了零化处理。

其他技术和策略：应用了Xgboost的MAE目标函数。对最新45天的数据使用了1.5倍的样本权重。 进行在线训练，整个过程中只重训练了两次模型。

训练代码：xgboost-training.ipynb

推理代码：https://www.kaggle.com/code/hookman/9th-submission（右上角copy and edit可以直接运行）

5.2 推理加速（选看）

原始特征列称为“字段”。我们将繁琐复杂的计算（例如 MACD、RSI、STD、SKEW 等）称为“算子（op）”。由算子组合而成的表达式称为“因子”。

一个因子要被视为阿尔法因子，需要展现出长期稳定的盈利能力，即与目标保持稳定的相关性。IC 指的是因子与目标随时间变化的相关性。IR 代表因子与目标相关性随时间变化的均值/方差比。评估一个因子是否为阿尔法因子，应考察其 IR 值是否相对较高。

在使用运算符生成因子时，如果使用原生 pandas，代码运行速度会非常慢。实际上，如果不是在 notebook 环境下，可以使用 C++ 进行编译。但在本次比赛的限制下，最佳方案仍然是使用 numba + multiprocessing 。

关于如何高效使用 Numba：使用 `numba.jit(nopython=True)` 。通过利用数学表达式原理，可以找到计算 NumPy 相关统计指标的公式。结合 `numba.prange` 和 `for` 循环可以实现一部分加速。对于特别复杂的数学公式，应该使用 `numba.jit(nopython=True, fastmath=True)` 。这样可以构建一个高效的时间序列统计方案。经验证，相比 pandas 提高20倍推理速度。对私有数据集的推理时间估计为 71 分钟（195 个特征），其中 150+ 个是滚动特征。单次推理耗时 1.04 秒，而优化前为 23.5 秒。

使用滚动特征时，用“流式缓存预测”，也就是在每次迭代时缓存数据输出，并将其连接起来以获取当前时间戳之前的特征，例如 ma(wap, 10) 。为了防止内存过载，可以指定一个最大更新时间戳来更新这个“缓存”。例如，对于每只股票，我们需要计算 60 天前的数据。

代码：https://www.kaggle.com/code/wenxuanxx/faster-function-build-feature

6 简历文字

项目背景：参与Optiver股票价格预测竞赛，基于纳斯达克订单簿和收盘拍卖数据，预测数百只股票的短期收盘价变动，评估指标为MAE。

个人职责：

- 特征工程体系构建：开发4大类动态特征，领域特征（买卖价差、流动性不平衡等）、交互特征（价格/交易量三重组合）、排序特征（按时间桶排名）、滚动特征（窗口统计量），特征总数达230+；实现CatBoost基于SHAP值的递归特征筛选，保留167个高贡献特征，本地CV的MAE降低0.15。
- 时间序列建模优化：融合LightGBM与CatBoost双模型优势，通过Hold-one-out和Purged CV差异化训练，设置训练-验证集时间Gap，杜绝未来信息泄露，相比传统K-Fold CV提升泛化性。
- 高性能计算优化：利用Numba加速交互特征计算，基于Optuna自动化超参搜索，优化LightGBM的叶子数（100-300）、学习率（0.005-0.02）等8项参数。经验证，相比 pandas 提高20倍推理速度。对私有数据集的推理时间估计为 71 分钟

（195 个特征），其中 150+ 个是滚动特征。单次推理耗时 1.04 秒，而优化前为 23.5 秒。

技术栈：LightGBM、CatBoost、Polars、Numba、Optuna、SHAP、时间序列交叉验证。